

データベース演習

第5回：要件定義と概念設計

鈴木源吾

前回の復習

1. 解説：概念設計とERモデル
 - モデルの表記について
2. 解説：ERモデルにおける関連
3. 解説：ER図の作図ツール

今回のトピック

1. 解説：要件定義と概念設計
2. 例題：関連とテーブル
3. 演習：関連とテーブル

1. 解説：要件定義と概念設計

この解説は演習には使わないので、参考扱い

帳票からのER図の作成

「帳票」とは，会社の取り引きや業務で使われる記録書類の総称

- 購入申請の伝票
- 旅費申請の伝票

システム化が進んでいないオフィスでは，Excelで作成された帳票が大量にある

→ そのような帳票を集め，必要な情報を抽出・整理し，システム化する

ボトムアップ分析とトップダウン分析

このような帳票などの既存の材料から、要件定義を進める分析手法を、**ボトムアップ分析**と呼ぶ

- 既存システムの画面やデータベースを利用する場合もある
- 取り組みやすいが、既存の見方に縛られ、あるべき姿を見失うリスクあり

反対に、既存の材料に頼ることなく、対象業務全体について、あるべき姿を検討しエンティティを抽出する手法を、**トップダウン分析**と呼ぶ

- あるべき姿に近づけるメリットがあるが、業務を知ることが難しい

→ **トップダウンで作ったモデルを、ボトムアップで補強するなど、両者の長所を合わせるのが理想的**

要件定義か設計か？

帳票からER図を作成する作業は、

- 要件を見つけて・整理するという意味では、**要件定義に含まれる**と言える
- データベースの概念設計を行うという意味では、**設計工程に含まれる**とも言える

ここは人やプロジェクトにより、様々な解釈があるが、整理の問題なので、両方正しいと考えてかまわない

帳票からER図を作成する手順

1. エンティティ（実体）の抽出
2. エンティティの属性の抽出
3. 主キーの決定
4. リレーションシップの作成
5. 多重度の決定

関連については前回説明しているので、1-2を説明する

例1：シラバス管理データベース

背景

N県のK大学では、シラバスがデータベース化されておらず、Excelによる帳票によって管理されていた。シラバスの検索や維持管理を容易にするため、シラバスデータベースを作成することにした。そのために、シラバスの帳票からER図の作成を行う

帳票の状況

- 科目毎に1枚の帳票が作成されている（帳票例は配布資料を参照）

単純化のための前提

- 本来はシラバスを複数年に渡って管理できる必要があるが（年毎にシラバスは変更可能性があり）、単純化のため、シラバスは年が変わっても変更がないとする

エンティティの抽出

注：データベースの基礎では、ER図の四角はエンティティタイプ（実体型）と呼んだが（ややアカデミックな呼び方）、情報処理技術者試験や実践的な参考書では、エンティティ（実体）と呼ぶことも多いため、今回はこのように呼ぶ。

エンティティについて

エンティティは、システムの管理対象となるものである。人・物・場所といった実体のあるものや概念などを選択する。システムの実装まで進むとデータベースのテーブルとして実装される

- 目安として、**リソース**（企業活動で必要となる人・物・場所・会社など）と**イベント**（企業活動を構成するアクティビティで、注文・発注・購買など）に分類して抽出することもある

シラバス帳票からのエンティティ抽出

シラバス帳票から，エンティティを抽出してみよう

- イベント系のエンティティはないが，リソース系はありそうだ
- 「もの」の繰り返しがあるところに，エンティティが存在する可能性がある．繰り返しがああるということは，それがテーブルになる可能性があるから
 - ここは第1正規化が意識されている．繰り返しがああるものは独立したテーブルにするのが第1正規化
 - 「管理すべき」繰り返しなのか，そうでないかは見極める必要がある

抽出例：「授業科目」

- この帳票全体は一つの授業科目を表しており，その概念を表すエンティティ
- このような帳票がいっぱいあるからエンティティとすることが適切

エンティティの抽出の例

授業科目に加えて、以下の3つのエンティティを抽出するのが、典型的な抽出例だろう

- 「单元」
 - 授業科目に対して1回の講義を表すエンティティである．繰り返しがあるのでエンティティとした
- 「使用図書」
 - 授業科目に対して複数の使用図書の指定が可能であり，その図書を表すエンティティ．これも繰り返しがあるためにエンティティとして独立させた
- 「担当教員」または「教員」
 - 教員は重要な情報なので，将来，授業科目以外の属性を管理する・授業科目を持たない教員も管理する，等の拡張をする可能性がある．よって「教員」エンティティを独立して作成する

エンティティの属性の抽出

1. 属性の候補になるものを見つける

- 「見出し」になっているものが属性になる可能性がある
- 「見出し」がなくても、値があるものには「見出し」が隠れている可能性がある

2. 属性がどのエンティティに属するかを決める

- 「見出し」に対する「値」が、どのエンティティによって決まるのかを見極める
- 「見出し」のない値が、どのエンティティによって決まるかを見極める

属性の抽出の例

見出しをリストアップする

- 担当教員名，対象学年，開講時期，・・・
- 概要，学習目標，単元・回数，授業計画又は学習の主題，・・・
- 書名，著者名，発行所
- 準備学習，評価方法，履修上の留意点

隠れている見出しをリストアップする

- 上記でリストアップした見出しとその値を除いて，残っているものを考える
→ 例えば，「データベース演習」が残る．これは授業科目名という見出しが隠れている

どのエンティティに属するかの決定

属性の値が、**どのエンティティの性質**を表しているか？と考える
(年齢という属性は、人間エンティティの性質を表す)

例：属性 概要は、どのエンティティに属するべきか？

- この概要には、授業科目の概要が書かれている → 概要は、授業科目の性質

→ 概要は、エンティティ 授業科目の属性にすることが適切である

エンティティが決まると属性の値は一意に決まるべきである

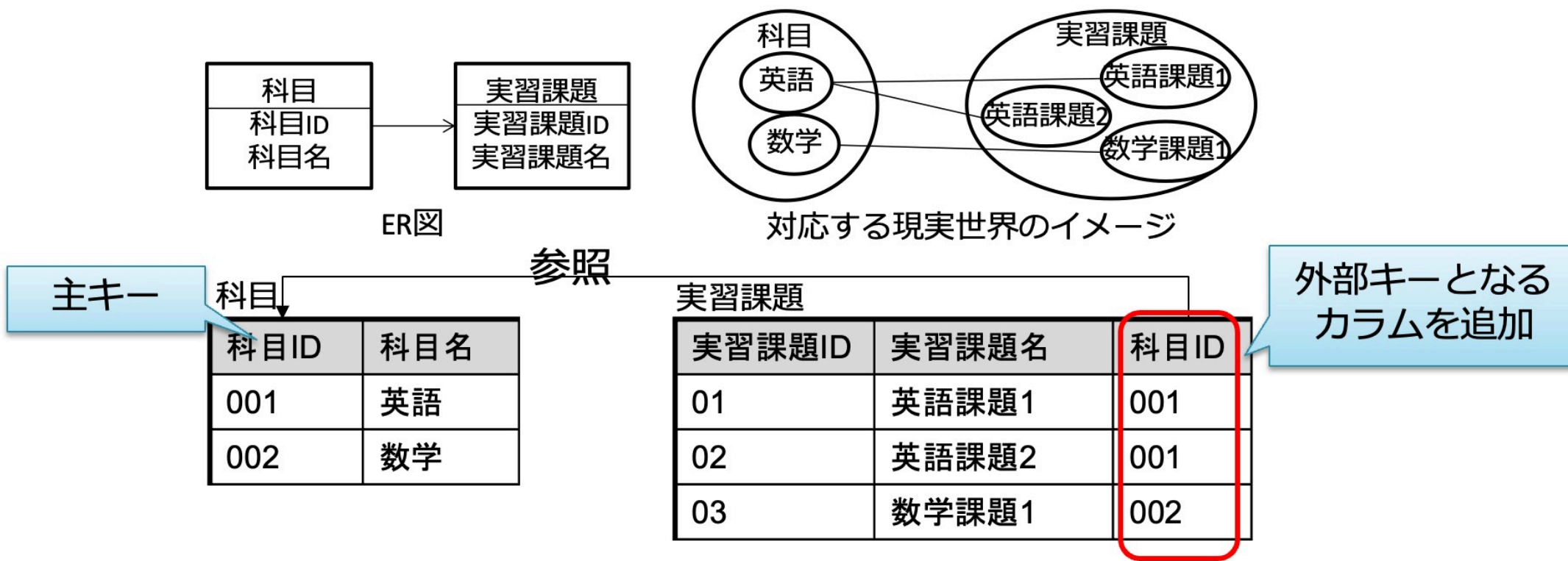
- これは、関数従属の考え方と同様である（また、キーと非キー属性の関係）

→ **属性の値を一意に決定できるエンティティを見極めればよい**

2. 例題：関連とテーブル

前回の復習：関連のテーブルへのマッピング（1対多）

- 下図のような主キーと外部キーの参照関係にマッピングすればよい
- 科目テーブル側に外部キーを入れるやり方はできない→一つのカラムに複数の実習課題IDを入れることができない＝第1正規形を壊す

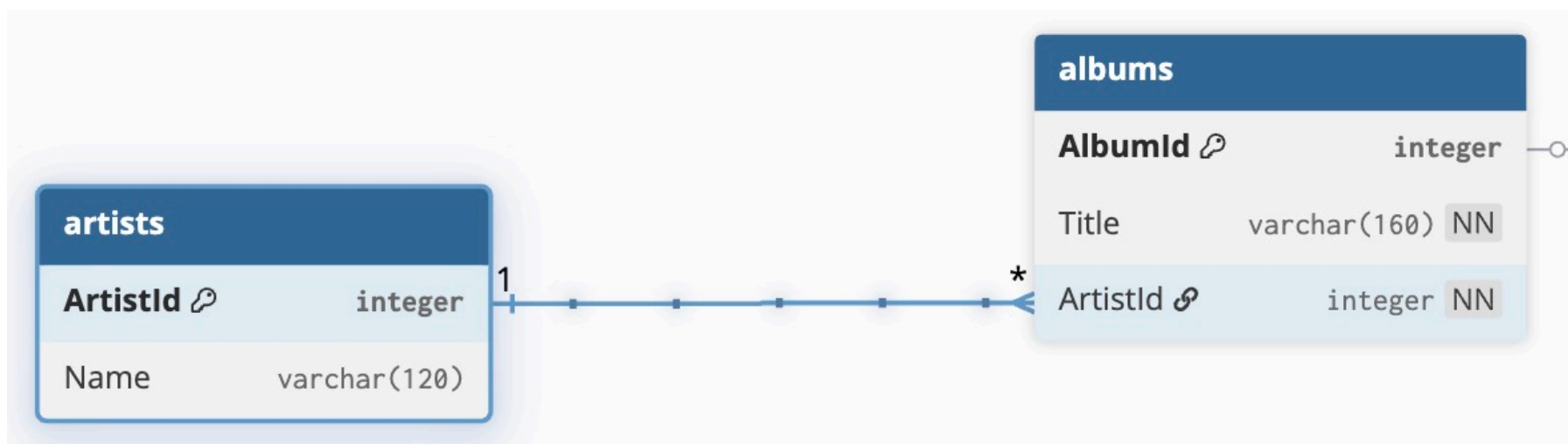


例題：ER図の関連とテーブルの関係を理解する

以下は，chinookデータベースのテーブルalbumsとartistsの関連をER図を使って表したものである(dbdiagram.ioを利用した)．以下の問いに答えよ．

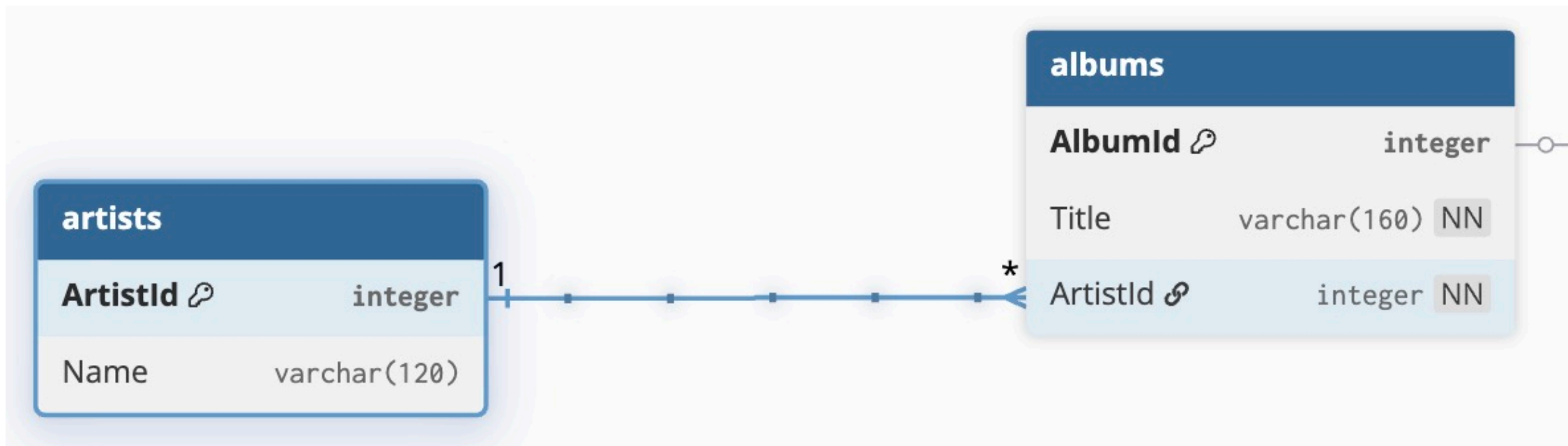
インタラクティブ版（dbdiagram.io）→ https://dbdiagram.io/d/chinook_sqlite-6a00b5c254a51d93d3e4f536

1. この関連の多重度は「1対多」であることがわかる．それでは，「多」の側であるテーブルはalbums・artistsのどちらか？



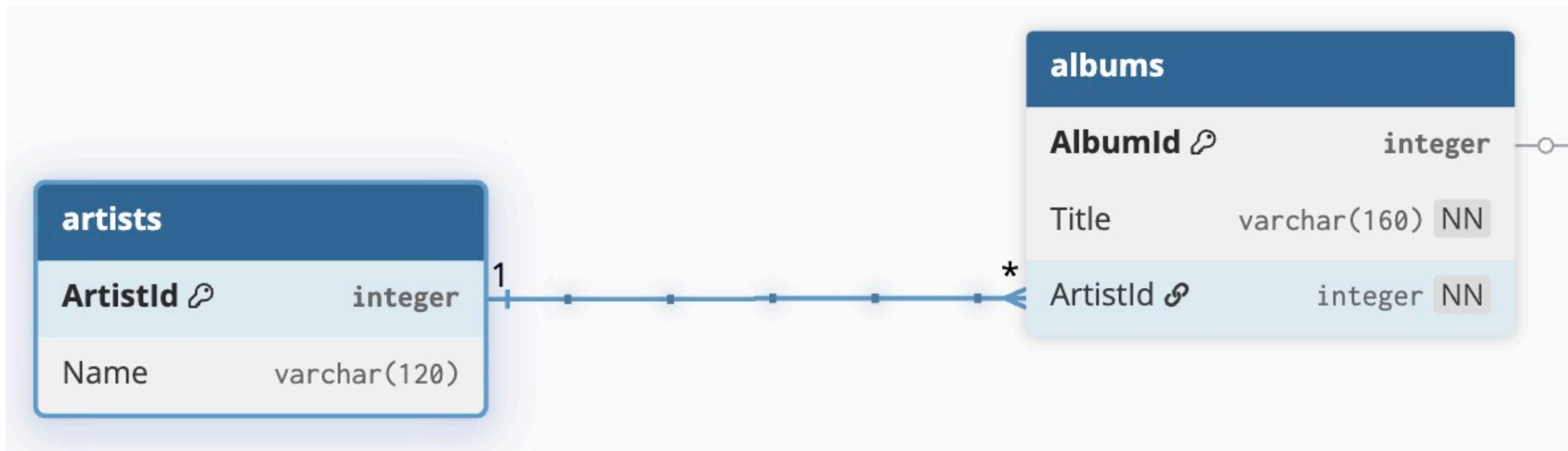
例題：ER図の関連とテーブルの関係を理解する

2. この関連は，テーブルの主キーと外部キーとの参照関係と対応している．
- (1) その主キーのあるテーブル名
 - (2) 主キーのカラム名
 - (3) その外部キーのあるテーブル名
 - (4) 外部キーのカラム名を，それぞれ答えよ



例題：ER図の関連とテーブルの関係を理解する

3. この関連は1対多となっているが，実際のchinookデータベースには，1つのデータに対して，2以上の個数のデータが関連づけられる場合はあるか？それを一目で確認できるSQL文を作成せよ．



例題の第3問のヒント：HAVING句

- 第3問に答えるためには，データベースの基礎で教わった内容を超える構文を使うと便利
→ **HAVING句**を使うと良い
- データベース演習では，SQLによる集計で再度説明予定
- 次ページ以降で説明

参考：HAVING句：グループ化した結果を絞り込む

集約結果をある条件によって絞り込みたいときがある

例) アクセス数が1よりも大きいユーザだけ、ユーザ毎のアクセス数を求めたい

WHERE句を使えばよい? → NG

- WHERE句は、GROUP BY句よりも先に実行されてしまう
 - WHERE句で絞り込んだ結果に対して、GROUP BY句が適用される
- 「WHERE count(*) > 1」と指定すると、文法エラーとなる

→ この目的のために使うのが**HAVING句**

- HAVING句は、GROUP BY句によってグループ化されたそれぞれのグループに対して条件を指定できる

参考：HAVING句の使用例

ユーザ毎のアクセス数を数えるSQL文



```
SELECT customer_id, count(*) FROM access_log_wide  
GROUP BY customer_id;
```

customer_idが20よりも大きいユーザについての、ユーザ毎のアクセス数を数えるSQL文



```
SELECT customer_id, count(*) FROM access_log_wide  
GROUP BY customer_id  
HAVING customer_id > 20;
```

アクセス数が1よりも大きいユーザについての、ユーザ毎のアクセス数を数えるSQL文



```
SELECT customer_id, count(*) FROM access_log_wide  
GROUP BY customer_id  
HAVING count(*) > 1;
```

元のテーブル

customer_id
18
53
18
18
32
53



HAVINGなしの結果

customer_id	count
18	3
53	2
32	1

HAVING customer_id > 20の結果

customer_id	count
53	2
32	1

HAVING count(*) > 1の結果

customer_id	count
18	3
53	2

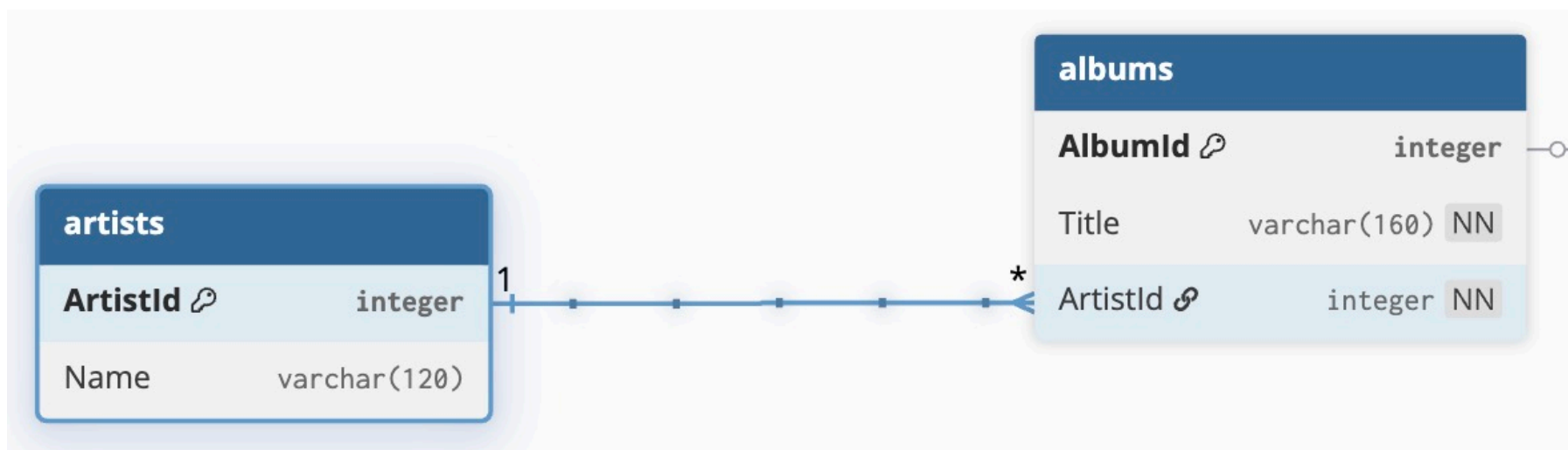
例題の解答例

1. この関連の多重度は「1対多」であることがわかる．それでは、「多」の側であるテーブルはalbums・artistsのどちらか？

答え：albums

（一人のアーティストに複数のアルバムが対応．アルバムが複数のアーティストに対応することはない）

解説：「鳥の足」の側が多．また，*も多を表す（UMLの記法）



例題の解答例

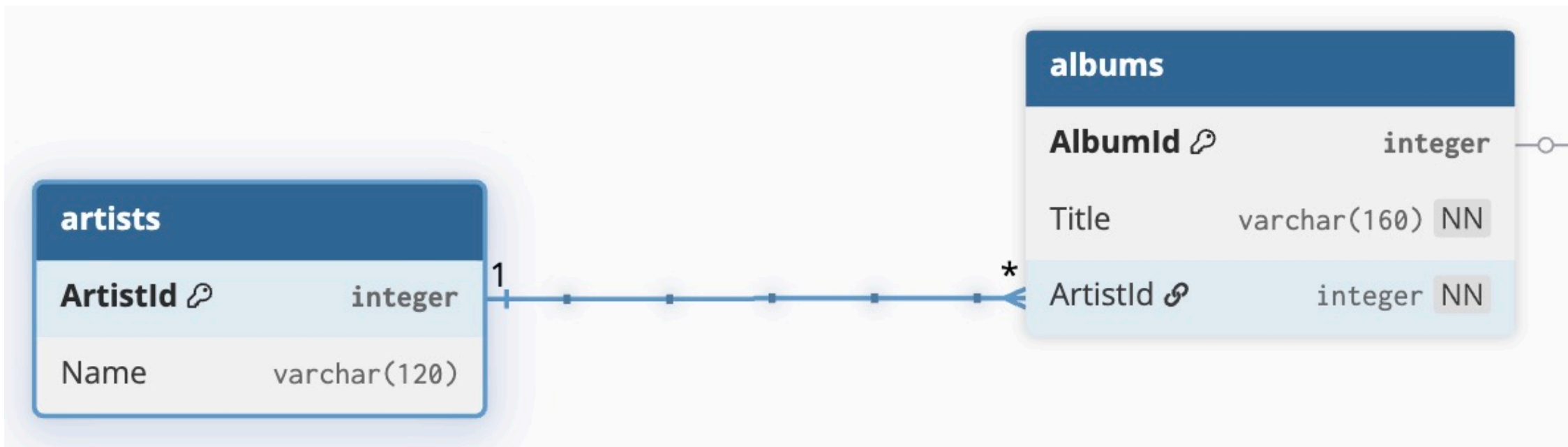
2. この関連は，テーブルの主キーと外部キーとの参照関係と対応している．

(1) その主キーのあるテーブル名 → 答え：artists

(2) 主キーのカラム名 → 答え：ArtistId

(3) その外部キーのあるテーブル名 → 答え：albums

(4) 外部キーのカラム名を，それぞれ答えよ → 答え：ArtistId



例題の解答例

3. この関連は1対多となっているが、実際のchinookデータベースには、1つのデータに対して、2以上の個数のデータが関連づけられる場合はあるか？それを一目で確認できるSQL文を作成せよ。

答え：以下のSQL文が複数の結果を返すため、1対多で2個以上に対応するデータはある。

(参考) 最も多いアルバムを出したアーティストはIron Maidenで21枚

逆に、1つのアルバムに複数のアーティストが対応している例はない

```
SELECT Name, count(*) FROM artists
JOIN albums ON artists.ArtistId = albums.ArtistId
GROUP BY artists.ArtistId HAVING count(*) >1;
```

3. 演習：関連とテーブル

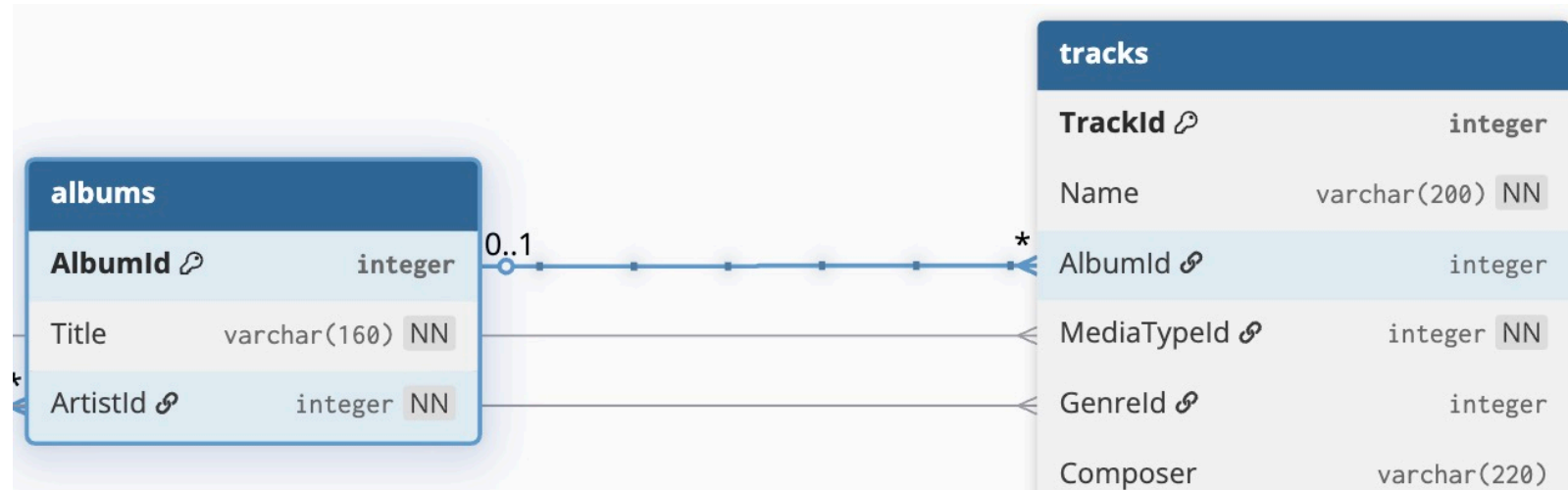
例題を参考にして実施しよう

演習：アルバムとトラックの関連

以下は，chinookデータベースのテーブルalbumsとtracksの関連をER図を使って表したものである(dbdiagram.ioを利用した)．以下の問いに答えよ．

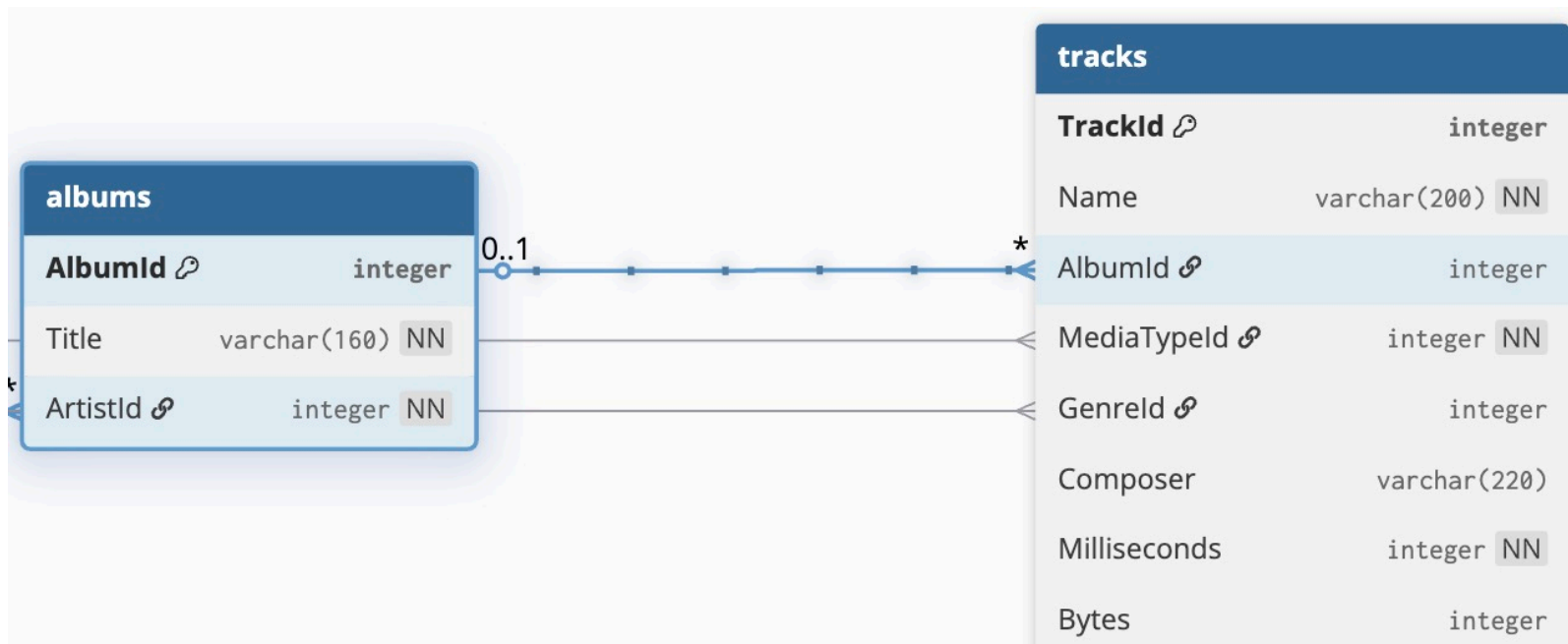
インタラクティブ版 (dbdiagram.io) → https://dbdiagram.io/d/chinook_sqlite-6a00b5c254a51d93d3e4f536

1. この関連の多重度は「1対多」であることがわかる．それでは，「多」の側であるテーブルはalbums・tracksのどちらか？



演習：アルバムとトラックの関連

2. この関連は，テーブルの主キーと外部キーとの参照関係と対応している．
- (1) その主キーのあるテーブル名
 - (2) 主キーのカラム名
 - (3) その外部キーのあるテーブル名
 - (4) 外部キーのカラム名を，それぞれ答えよ



演習：アルバムとトラックの関連

3. この関連は1対多となっているが、実際のchinookデータベースには、1つのデータに対して、2以上の個数のデータが関連づけられる場合はあるか？それを一目で確認できるSQL文を作成せよ。
4. 前の問題に関連して、実際のchinookデータベースを用いて以下を求めて答えよ。
1つのデータに対して、2以上の個数のデータが関連づけられているときの、その個数の最大値を求めよ